



**Tecnológico
de Monterrey**

Evidencia1:

**Conceptos Básicos y Algoritmos
Fundamentales**

Allison Charlize Arriaza - A00844457

Profesor: David Alonso Cantú

20 de septiembre, 2026

Av. Eugenio Garza

Sada 2501 Sur, Tecnológico

64700 Monterrey, N.L.

**Programación de estructuras de datos y algoritmos
fundamentales**

(Grupo 607)

Reflexión

En esta evidencia pude comparar de una manera práctica cómo funcionan distintos algoritmos de ordenamiento y cómo cambia su rendimiento dependiendo de los datos que reciben. Al trabajar con archivos de logs, me di cuenta que tener la información ordenada no solo sirve para organizar los registros, sino también para hacer búsquedas de una manera más eficiente.

Durante la actividad se utilizaron Swap Sort, Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Merge Sort y Quick Sort. Aunque todos buscan ordenar los datos, sus tiempos fueron bastante diferentes. Por ejemplo, Insertion Sort tardó 4,138,799 microsegundos en log607-1.txt, pero solamente 45,182 microsegundos en log607-2.txt. Esto pasó porque el segundo archivo ya estaba casi ordenado, por lo que el algoritmo necesitó hacer menos movimientos.

Sin embargo, se notó una diferencia importante con Quick Sort. En log607-1.txt tardó 100,973 microsegundos, mientras que en log607-2.txt tardó 2,580,467 microsegundos. Esto probablemente se deba a la manera en que se eligió el pivote, ya que se utiliza el último elemento del arreglo. Por lo que cuando los datos ya están casi ordenados, ese pivote puede generar divisiones poco equilibradas y hacer que el algoritmo tenga que realizar más comparaciones.

Merge Sort y Shell Sort también tuvieron buenos resultados en los dos archivos. Sus tiempos fueron bastante parecidos en ambas pruebas, a diferencia de otros algoritmos donde la diferencia fue mayor dependiendo del archivo. Con esto, entendí que la complejidad teórica ayuda a entender el rendimiento, pero también influye cómo vienen ordenados los datos.

Tener los registros ordenados también permitió utilizar búsqueda binaria para encontrar un rango de fechas. En lugar de revisar todos los registros uno por uno, este método va reduciendo el espacio de búsqueda. Esto hizo más eficiente la localización de la fecha inicial y final dentro de los 6818 registros.

Uso de la IA

Durante esta evidencia utilicé inteligencia artificial cuando tenía dudas específicas sobre algunas partes del programa. La usé principalmente para revisar la lógica de ciertas funciones y para entender mejor cómo podía adaptar algunas ideas al código que ya tenía hecho.

Algunos de los prompts que utilicé fueron: “¿Cómo puedo comparar fechas guardadas dentro de un struct en C++?”, “¿Cómo puedo hacer una búsqueda binaria para encontrar el inicio y el final de un rango de fechas?” y “¿Cómo puedo medir el tiempo de ejecución de los algoritmos usando chrono?”. A partir de las respuestas fui haciendo cambios para que funcionaran con mi estructura, especialmente con Registro, compFechas y el vector donde se guardaban los datos.

También hubo sugerencias que tuve que corregir. En la búsqueda binaria para encontrar el final del rango, una de las propuestas actualizaba mal los límites left y right. Eso hacía que la búsqueda no llegara correctamente al último registro que todavía estaba dentro de la fecha final. Me di cuenta al probar con fechas reales y ver que el rango que regresaba no coincidía con lo que esperaba.

Para corregirlo revisé qué debía pasar en cada caso. Si el registro en mid todavía estaba dentro del rango, tenía que guardar esa posición y mover left a $\text{mid} + 1$ para seguir buscando hacia la derecha. Si ya se pasaba de la fecha final, entonces debía mover right a $\text{mid} - 1$.

Esta parte fue de las más importantes para mí porque, si no hubiera hecho pruebas con fechas reales, probablemente habría dejado una búsqueda que regresaba un rango incompleto. Después de eso fui revisando con más cuidado cada sugerencia antes de dejarla en la versión final del programa.